

Step A: switch the live app to Solana devnet

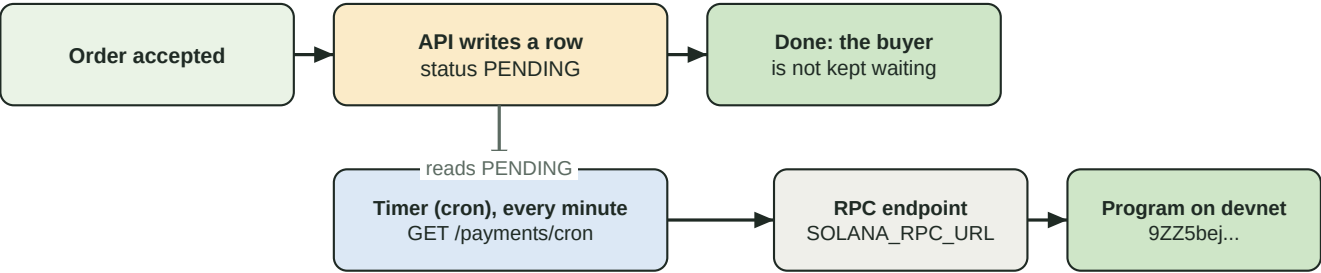
This is the last part of Phase 1. The code is done and tested. What is left is settings, not programming. Nothing here moves real money: devnet coins are free test coins.

What changes, in one picture

BEFORE (today): the mock provider answers instantly, nothing leaves our server



AFTER (Step A): the row waits, a timer sends it to Solana devnet



When it lands the row turns CONFIRMED and stores the transaction signature you can open in the explorer.

Figure 1. The buyer never waits for the blockchain. The API only writes a note; a timer delivers it.

What stays the same

- Orders, listings, matching and every page of the app behave as before.
- If Solana is slow or down, orders still go through. The failed note is kept and retried.
- Orders accepted before the switch have no commit note, so nothing is sent for them. That is intended.

Part 1: what is already done

Item	Status	Detail
Backend signing key	Created	Public address 8dxtAKX8UUDCeuKbtr6foBXedV7ZU5X23iUXR9TAacA. File: ~/.config/solana/farmers-market-backend-devnet.json (permissions 600, outside the git repo). Devnet only.
Funding	1 SOL on devnet	Sent from your laptop wallet. Each order needs about 0.00125 SOL of rent plus 0.000005 SOL per step.
Program	Deployed	9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU, 13 of 13 smoke checks pass on devnet.
Queue and timer route	In the live code	Pushed to GitHub and deployed. The route stays locked (401) until CRON_SECRET exists.
The five settings on Vercel	NOT set	Blocked by the safety rules of my tool, because it means writing secrets to your Vercel account. You do this part.

Part 2: add the five settings

The five settings and what each one does

PAYMENT_PROVIDER = solana	the switch: use the chain instead of the mock
SOLANA_PROGRAM_ID	which program to talk to
SOLANA_RPC_URL	which door to the network (public, or a keyed one)
SOLANA_AUTHORITY_KEYPAIR_JSON	the backend's signing key. SECRET.
CRON_SECRET	password the timer must send. SECRET.

Red = a leaked value lets someone act as your backend. Mark both Sensitive in Vercel.

Figure 2. Three plain settings, two secrets.

Name	Value	Sensitive?
PAYMENT_PROVIDER	solana	no
SOLANA_PROGRAM_ID	9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU	no
SOLANA_RPC_URL	https://api.devnet.solana.com (works, but rate-limited; a keyed URL from Helius or similar is steadier)	no
SOLANA_AUTHORITY_KEYPAIR_JSON	the whole contents of the key file: a list of 64 numbers in square brackets	yes
CRON_SECRET	a long random string you make up	yes

Choose one way to add them

Way 1: Vercel dashboard (easiest to see)

1. Open vercel.com, project **farmers-market-place** (the backend, not the -le5n one).
2. Settings, then Environment Variables.
3. For each row in the table: type the name and value, tick **Production**, and for the two secrets tick **Sensitive**. Save.
4. Get the key contents in a terminal with the command below, copy the whole line, paste it as the value.

```
$ cat ~/.config/solana/farmers-market-backend-devnet.json  
[12,200,...64 numbers...,31]  
# make a cron secret: run this, copy the output  
$ python3 -c "import secrets;print(secrets.token_urlsafe(32))"
```

Way 2: one command per setting, from your terminal

Type each line into the Claude Code prompt starting with the exclamation mark, so it runs in this session with your approval. The value goes in through the pipe, so it is not typed into the command itself.

```
$ cd ~/Dev/singapore/api  
$ printf '%s' solana | npx vercel env add PAYMENT_PROVIDER production --project farmers-market-place  
$ printf '%s' 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU | npx vercel env add SOLANA_PROGRAM_ID production --  
$ printf '%s' https://api.devnet.solana.com | npx vercel env add SOLANA_RPC_URL production --project farmers-ma  
$ cat ~/.config/solana/farmers-market-backend-devnet.json | npx vercel env add SOLANA_AUTHORITY_KEYPAIR_JSON pr  
$ printf '%s' YOUR_RANDOM_STRING | npx vercel env add CRON_SECRET production --project farmers-market-place --s
```

Check the names landed (values stay hidden):

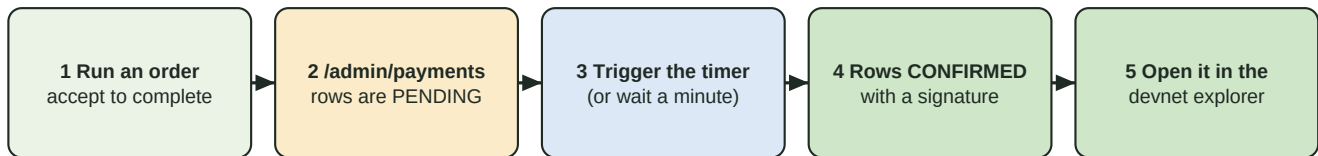
```
$ npx vercel env ls production --project farmers-market-place | grep -E 'SOLANA|CRON|PAYMENT'
```

Then redeploy

Settings only apply to a new deployment. In the Vercel dashboard open the backend project, Deployments, the latest one, the three-dot menu, **Redeploy**. Wait for Ready.

Part 3: prove it works

How you know it worked: follow one order from the app to the explorer



Success = every row of a finished order is CONFIRMED: one commit, one per attestation, one settle.

A row that says FAILED is not a disaster: the reason is stored and the next tick retries it.

Figure 3. One order, followed all the way to the explorer.

Run these after the redeploy is Ready. Replace the two placeholders.

```
# get an admin token (this reseeds the demo data: fine while there are no real users)
$ curl -s -X POST 'https://farmers-market-place-nine.vercel.app/demo/seed?role=admin'
# in the web app: log in as buyer, create an order; farmer accepts; run it to COMPLETED
# then look at the notes:
$ curl -s -H 'Authorization: Bearer ADMIN_TOKEN' https://farmers-market-place-nine.vercel.app/admin/payments
[{"purpose":"order","status":"PENDING","provider":"solana",...}]
# trigger the timer by hand instead of waiting
$ curl -s -H 'Authorization: Bearer YOUR_CRON_SECRET' https://farmers-market-place-nine.vercel.app/payments/cron
{"processed":5}
```

Run the admin call again: the rows say CONFIRMED and each has an external_id. That is the transaction signature. Open it:

```
$ https://explorer.solana.com/tx/<external_id>?cluster=devnet
```

You should see Result: Success and in the logs Instruction: CommitOrder, RecordAttestation or Settle. The backend key address in the page is the one in Part 1.

If something looks wrong

What you see	Meaning	Fix
Timer call returns 401	CRON_SECRET missing, wrong, or you did not redeploy	Check the name, re-add, redeploy, resend with the exact value
Rows stay PENDING	Nothing has called the timer yet	Call /payments/cron by hand, or schedule it (see below)
FAILED, 'getSignatureStatuses 429'	The public endpoint is rate-limiting	Call the timer again, or use a keyed RPC URL
FAILED, insufficient funds	Backend key is out of devnet SOL	<code>solana -u devnet transfer 8dxtAKX8UUDCeUkb ttr6foBXedV7ZU5X23iUXR9TAacA 1</code>
FAILED, 'AccountNotFound' on attest or settle	That order's commit step has not landed yet	Nothing: the next tick replays commit first, then the rest, in order
Order screens show a 500	Database is missing migration 0004	alembic upgrade head with the unpooled Neon URL

Run the timer automatically

A recurring call to /payments/cron with the header `Authorization: Bearer YOUR_CRON_SECRET`. Two options:

- **Vercel Cron** (needs a Pro plan for a once-a-minute schedule; Hobby only allows daily and rejects the whole deploy otherwise). I will add the schedule once you tell me your plan.
- **Any outside scheduler** that can send that header every minute, for example a GitHub Actions schedule.

Undo, in two minutes

Change `PAYMENT_PROVIDER` back to mock (or delete it) and redeploy. New orders go back to instant mock rows. Notes already stored stay in the database and do no harm.

Done when: one finished order shows `CONFIRMED` rows in /admin/payments and each signature opens as a Success transaction on the devnet explorer. That closes Phase 1; Stage 2 (token escrow) comes next.